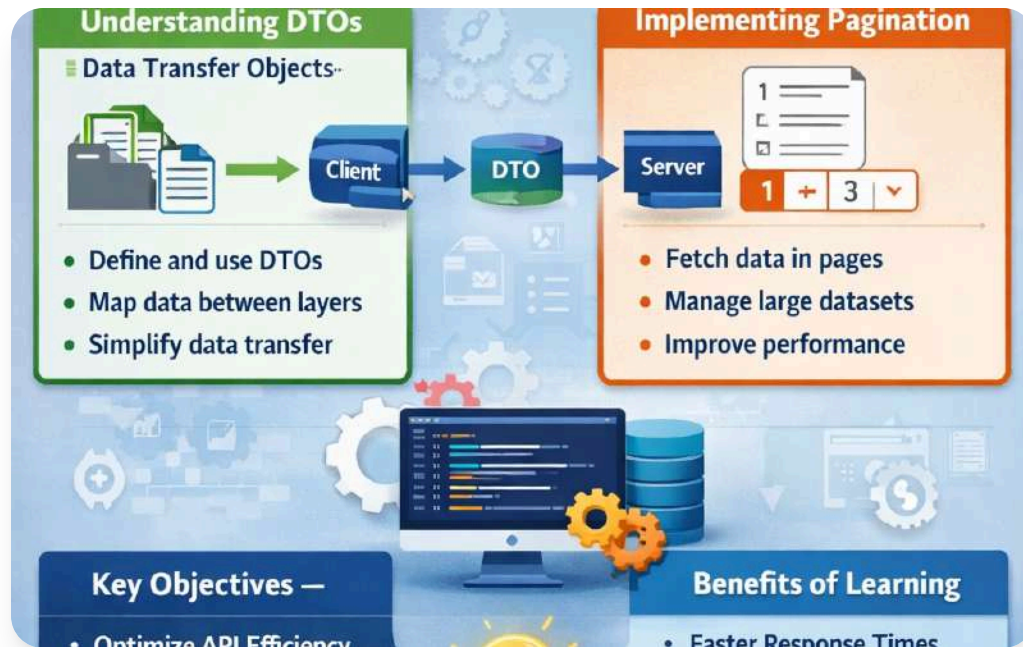




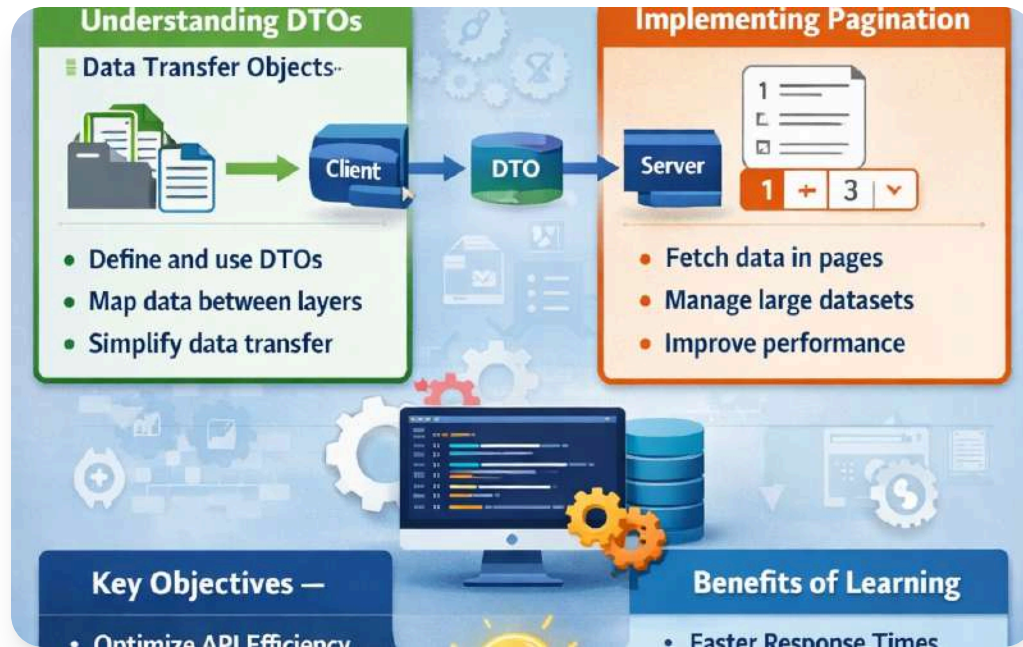
Golang Gin Backend Pertemuan 15

Membahas DTO request dan response, format API response konsisten, pagination, serta fitur search pada backend Golang Gin.



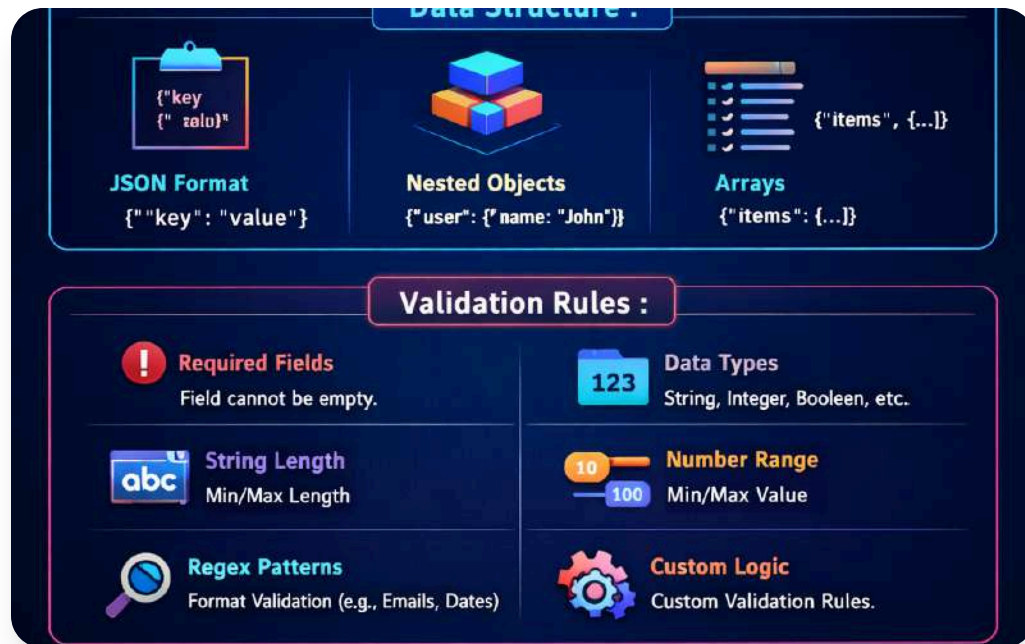
Juara Coding

June 2026



Tujuan Pembelajaran

Peserta memahami fungsi DTO, menyusun response API rapi, menerapkan pagination, dan menambah parameter pencarian.



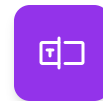
Konsep DTO pada Backend Gin

DTO mendefinisikan data dari client dan server agar validasi lebih terarah, field sensitif tersembunyi, dan kontrak API lebih jelas.

DTO Request

Field	Type	Validation
userId	Integer	None
username	String	None
email	String	None
age	Integer	None
active	Boolean	None
createdAt	DateTime	None

DTO request merepresentasikan data input dari client untuk create, update, dan parameter endpoint dengan validasi.



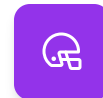
Input dari Client

Digunakan untuk menampung data yang dikirim client saat membuat atau memperbarui data.



Validasi Request

Biasanya memakai tag json dan binding agar struktur input sesuai kebutuhan endpoint.



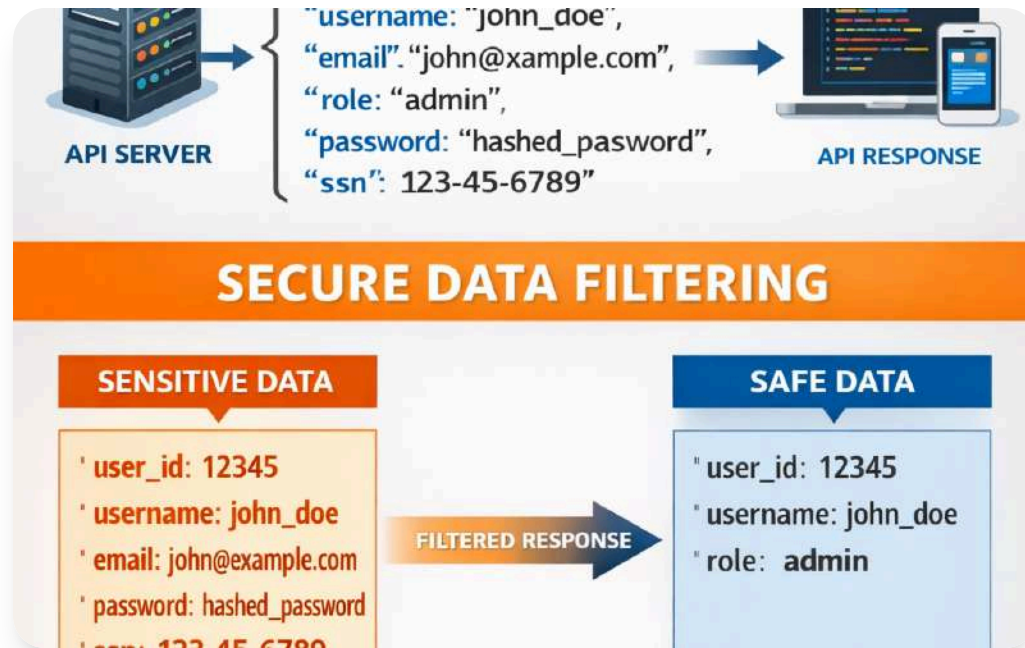
Field Sesuai Endpoint

Contohnya name, email, page, limit, atau keyword tergantung fungsi endpoint yang dibuat.

Contoh DTO Request di Gin

Struct	Sumber Data	Field Utama	Validasi/Tag
CreateUserRequest	JSON body	Name, Email	required, email
UserListRequest	Query parameter	Page, Limit, Search	form tag

DTO ini membantu proses bind data dari body JSON maupun query parameter pada Gin agar input lebih rapi dan mudah divalidasi.



DTO Response

DTO response menentukan data yang dikirim ke client agar field internal tetap terlindungi dan output antar endpoint tetap konsisten.

Contoh DTO Response

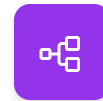
Field	Type	JSON Tag
ID	uint	id
Name	string	name
Email	string	email

Response DTO hanya menampilkan data yang relevan untuk client. Field sensitif seperti password tidak disertakan agar output lebih aman dan rapi.

API Response yang Konsisten



Format response yang seragam memudahkan frontend, integrasi, error handling, dokumentasi, dan debugging.



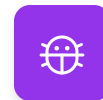
Struktur Seragam

Response umumnya berisi status, message, data, dan metadata opsional.



Integrasi Lebih Mudah

Frontend dan sistem lain lebih mudah membaca serta memproses response.



Debugging Cepat

Pola yang konsisten membuat penanganan error dan pelacakan masalah lebih jelas.

Contoh Struktur API Response

Field	Type	Keterangan
Status	string	Menunjukkan status response
Message	string	Pesan hasil proses
Data	interface{}	Payload opsional sesuai kebutuhan
Metadata	opsional	Dapat ditambah untuk pagination

Tabel ini merangkum field utama pada struktur response API yang dapat digunakan untuk kondisi sukses maupun gagal, serta dapat diperluas dengan metadata tambahan.

Contoh Response API

200 OK

```
{
  "status": "success",
  "data": {
    "userId": 12345,
    "name": "John Doe",
    "email": "johndoe@example.com"
  }
}
```

404 Not Found

```
{
  "status": "error",
  "message": "User not found",
  "code": 404
}
```

Format response dibuat seragam untuk kondisi sukses maupun error agar endpoint lebih konsisten dan mudah digunakan.



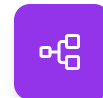
Response sukses

Mengembalikan status, pesan, dan data saat proses berjalan dengan baik.



Response error

Mengembalikan status dan pesan yang jelas saat input atau proses tidak valid.



Format seragam

Struktur yang konsisten memudahkan integrasi, debugging, dan pemeliharaan endpoint.



No Data Available

There are no records to display.

Pagination pada Endpoint List

Pagination membatasi data dengan parameter *page* dan *limit* agar respons lebih ringan, cepat, dan efisien untuk tampilan list di client.

Parameter Pagination

API Pagination Parameters			
page	limit	offset	None
Current page number	Number of items per page	Starting position of records	No pagination applied

01 Parameter utama

Parameter umum adalah *page* untuk nomor halaman dan *limit* untuk jumlah data per halaman.

02 Rumus offset

Nilai *offset* biasanya dihitung dengan rumus $(page - 1) \times limit$ untuk menentukan titik awal data.

03 Contoh penggunaan

Jika *page* = 2 dan *limit* = 10, maka data diambil mulai dari *offset* 10.

Implementasi Pagination di Gin

Komponen	Kode/Peran
page	Ambil query page, default 1
limit	Ambil query limit, default 10
offset	Hitung $(page - 1) * limit$
Query DB	Gunakan <code>Offset(offset).Limit(limit)</code>
Hasil	Data users diambil sesuai halaman

Contoh ini menunjukkan cara sederhana membatasi data per halaman di Gin menggunakan parameter query dari client dan pagination pada database.

Metadata Pagination pada Response

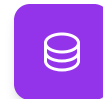
```
{  
  "total": null,  
  "per_page": null,  
  "current_page": null,  
  "last_page": null,  
  "next_page": null,  
  "prev_page": null,  
  "has_more": null  
},
```

Tambahkan metadata pagination agar client memahami posisi halaman, total data, dan ketersediaan halaman berikutnya.



Posisi Halaman

Field seperti page dan limit membantu client mengetahui halaman aktif dan ukuran data per request.



Total Data

Informasi total_data dan total_page memberi gambaran jumlah keseluruhan data yang tersedia.



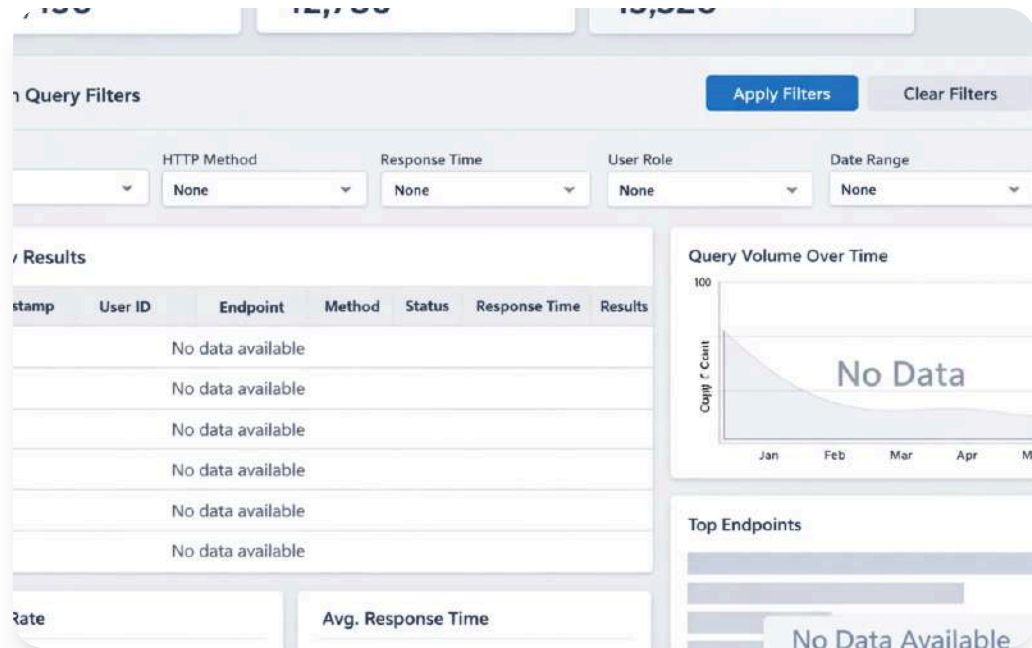
Navigasi Client

Metadata memudahkan client menentukan apakah masih ada halaman berikutnya untuk diambil.

Contoh Response Pagination

Field	Contoh Nilai
status	success
message	list user berhasil diambil
data	users
meta.page	page
meta.limit	limit
meta.total_data	totalData

Struktur response ini umum dipakai pada endpoint list, dengan data utama dan informasi pagination di dalam field meta.



Search pada Endpoint API

Fitur **search** membantu client memfilter data dengan kata kunci, seperti nama atau email, dan idealnya dipadukan dengan pagination.

Contoh Implementasi Search

Bagian	Implementasi
Ambil keyword	<code>search := c.Query("search")</code>
Inisialisasi query	<code>query := db.Model(&User{})</code>
Filter pencarian	<code>query.Where("name LIKE ? OR email LIKE ?", ...)</code>
Eksekusi	<code>query.Find(&users)</code>

Contoh ini menunjukkan alur search sederhana: mengambil keyword, menambahkan filter bila ada input, lalu mengembalikan data yang cocok.

DTO, Pagination, dan Search

```
"totalItems": 25,  
"search": "example",  
"category": null,  
"items": [  
  {  
    "id": 1,  
    "name": "Item One",  
    "description": "This is the first item.",  
    "price": 19.99  
  },  
  {  
    "id": 2,  
    "name": "Item Two",  
    "description": "This is the second item.",  
    "price": 29.99  
  }  
]
```

01 Bind query ke DTO request

Pada endpoint list, query parameter dibind ke DTO request agar input lebih terstruktur dan mudah divalidasi.

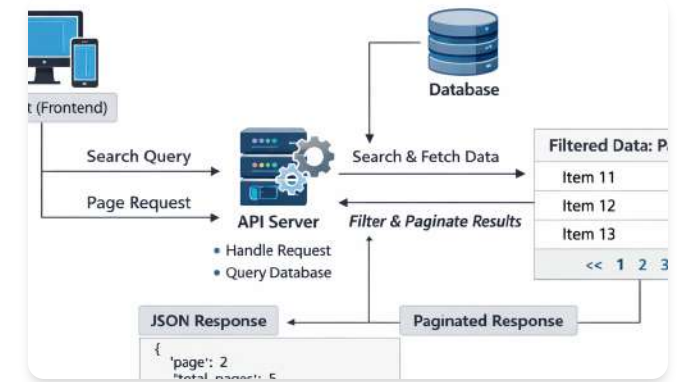
02 Proses pagination dan search

DTO request lalu digunakan untuk mengatur pagination dan search sehingga logika list tetap rapi dan konsisten.

03 Kirim DTO response konsisten

Hasil data dikonversi ke DTO response dan dikirim dalam format API response yang konsisten agar mudah dirawat.

Alur Endpoint List User



01 Request dari Client

Client mengirim parameter *page*, *limit*, dan *search* ke endpoint list user.

02 Proses di Backend

Gin melakukan binding ke DTO request, lalu backend memvalidasi input sebelum menjalankan query database.

03 Response dan Metadata

Hasil query dipetakan ke DTO response dan dikirim bersama metadata pagination.

Best Practice Implementasi



Pisahkan DTO Request dan Response

Gunakan DTO berbeda agar kontrak input dan output lebih jelas serta mudah dipelihara.



Atur Pagination dan Search dengan Aman

Tetapkan default page-limit dan batasi search hanya pada field yang relevan dan aman di-query.



Standarkan Validasi dan Response

Validasi input dengan binding dan gunakan helper response agar format API konsisten.

The infographic is divided into two main sections: 'Best Practices' and 'Checklist'. The 'Best Practices' section lists seven items, each with an icon: 'Use RESTful & JSON Standards' (REST icon), 'Secure Endpoints (OAuth, API Keys)' (shield with key icon), 'Rate Limiting & Throttling' (speedometer icon), 'Detailed API Documentation' (notepad icon), 'Versioning & Deprecation Policy' (v1 and arrow icon), and 'Error Handling & Logging' (warning triangle icon). The 'Checklist' section lists six items, each with a green checkmark icon: 'Input Validation & Sanitization.', 'Authentication Testing,', 'Rate Limit Testing,', 'Error Response Testing,', 'Performance & Load Testing,', and 'Security Testing (SSL, Penetration Tests)'. At the bottom, there is a blue banner with the text 'BUILD SECURE & RELIABLE APIs' and an icon of a shield with a checkmark. The word 'API' is also visible on the left side of the banner.

Best Practices	Checklist
Use RESTful & JSON Standards	☒ Input Validation & Sanitization.
Secure Endpoints (OAuth, API Keys)	☒ Authentication Testing,
Rate Limiting & Throttling	☒ Rate Limit Testing,
Detailed API Documentation	☒ Error Response Testing,
Versioning & Deprecation Policy	☒ Performance & Load Testing,
Error Handling & Logging	☒ Security Testing (SSL, Penetration Tests)

API BUILD SECURE & RELIABLE APIs

Kesalahan yang Sering Terjadi



Model DB Dipakai Langsung

Hindari memakai model database langsung sebagai request atau response API.



Search Tanpa Batasan

Pencarian tanpa batasan membuat query lambat dan sulit dioptimalkan.



Validasi dan Metadata Terlewat

Query parameter tidak divalidasi, total data tidak dihitung, dan metadata pagination sering terlupa.

```
{
  "error": "Invalid pagination parameters",
  "message": "Page number must be a positive integer.",
  "code": 400
}
```

```
{
  "error": "Page out of range",
  "message": "Requested page 50 exceeds total pages: 10",
  "code": 400
}
```

Inconsistent Pagination Data

```
{
  "page": 3
  "total_pages": 5,
  "results": [],
  "b,
  "No results found; ~page indicates mesauess out of range!"
}
```

Missing Required Parameter

```
{
  "error": "Missing parameter",
  "message": "The \"limit\" parameter is required.",
  "code": 422
}
```

Latihan Praktik 1

01 Endpoint GET /products

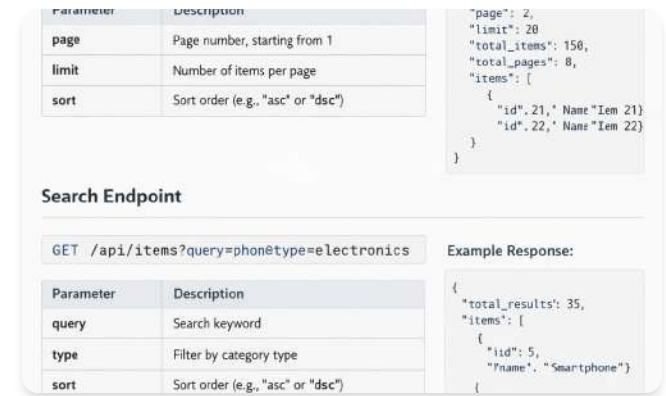
Buat endpoint GET /products dengan dukungan query parameter page, limit, dan search untuk kebutuhan daftar produk.

03 Gunakan DTO terpisah

Pisahkan DTO request dan DTO response agar validasi input dan struktur output lebih jelas dan mudah dikelola.

02 Search dan format response

Pencarian minimal mencakup field name dan category. Response wajib berformat status, message, data, dan meta.



Latihan Praktik 2

01 POST /customers

Buat endpoint untuk menambahkan data customer menggunakan DTO request sebagai format input yang tervalidasi.

03 Pagination dan Search

Tambahkan pagination serta pencarian berdasarkan **name** atau **email** pada endpoint list customer.

02 GET /customers

Tampilkan daftar customer dengan DTO response dan format API response yang konsisten untuk setiap hasil.

The screenshot shows an API client interface. At the top, there is a URL bar with the text "https://api.example.com/customers/{customer_id}" and a "Send" button. Below the URL bar, there is a section for parameters. It has a table with columns "Name", "Value", and "Type". The first row has "customer_id" in the Name column, "12345" in the Value column, and "Path" in the Type column. There is a "+ Add Param" link to the right of the table. Below the parameters section, there is a "Request Body" section with tabs for "JSON" and "Form Data". The "JSON" tab is selected. At the bottom, there is a "Response" section with a "+ Add M..." link.

Name	Value	Type
customer_id	12345	Path



Ringkasan Materi

DTO, response konsisten, pagination, dan search penting untuk backend Gin yang rapi dan scalable.